

Práctico 4 – Bruno González – Base de Datos 2

Profesor: Cesar Augusto Barrionuevo Saavedra

- 1) Enunciado: Crear una función `precio_final(p_id_producto INT)` que busque el precio del producto en la tabla `productos` y retorne el precio más 21% de IVA. Luego usarla en un `SELECT` que liste todos los productos con su precio base y su `precio_final`.

-- Ejercicio 1

```
DELIMITER $$
CREATE FUNCTION precio_final(p_id_producto INT)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE variable_preco DECIMAL(10,2);
    SELECT precio INTO variable_preco FROM productos WHERE id_producto = p_id_producto;

    RETURN COALESCE(variable_preco * 1.21, 0);
END$$
DELIMITER ;

SELECT p.precio, precio_final(p.id_producto) AS precio_final FROM productos p;
```

- 2) Enunciado: Crear una función `dias_como_cliente(p_id INT)` que retorne cuántos días han pasado desde `fecha_registro` del cliente hasta hoy. Usarla en un `SELECT` para listar clientes ordenados por antigüedad (más antiguo primero).

-- Ejercicio 2

```
DELIMITER $$
CREATE FUNCTION dias_como_cliente(p_id INT)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE variable_fecha_registro DATETIME;
    SELECT fecha_registro INTO variable_fecha_registro FROM clientes WHERE id_cliente = p_id;

    RETURN DATEDIFF(NOW(), variable_fecha_registro);
END$$
DELIMITER ;

SELECT c.id_cliente, c.nombre, dias_como_cliente(c.id_cliente) AS dias_como_cliente FROM clientes c ORDER BY dias_como_cliente(c.id_cliente) DESC;
```

- 3) Enunciado: Crear un procedure registrar_cliente_completo(nombre VARCHAR, email VARCHAR, saldo_inicial DECIMAL, tipo_cuenta ENUM) que, dentro de una transacción, inserte el cliente en clientes y luego inserte su cuenta en cuentas con el saldo inicial. Si cualquier paso falla, hacer ROLLBACK completo.

-- Ejercicio 3

```
DELIMITER $$
CREATE PROCEDURE registrar_cliente_completo(
  IN new_nombre VARCHAR(150),
  IN new_email VARCHAR(150),
  IN new_saldo_inicial DECIMAL(12,2),
  IN new_tipo_cuenta ENUM('corriente', 'ahorro')
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
  START TRANSACTION;
  INSERT INTO clientes(nombre, email, saldo) VALUES(new_nombre, new_email, new_saldo_inicial);
  INSERT INTO cuentas(id_cliente, tipo, saldo) VALUES(LAST_INSERT_ID(), new_tipo_cuenta, new_saldo_inicial);
  COMMIT;
END$$
DELIMITER ;

-- Para corroborar el rollback
CALL registrar_cliente_completo('Juan Pérez', 'juan@example.com', 1000, 'ahorro');
```

- 4) Enunciado: Crear un procedure resumen_cliente(IN p_id INT, OUT p_total_ventas INT, OUT p_monto_total DECIMAL(12,2)) que cuente cuántas ventas realizó un cliente y cuánto dinero gastó en total. Llamarlo con CALL y mostrar ambos valores con SELECT @var1, @var2.

-- Ejercicio 4

```
DELIMITER $$
CREATE PROCEDURE resumen_cliente(
  IN p_id INT,
  OUT p_total_ventas INT,
  OUT p_monto_total DECIMAL(12,2)
)
BEGIN
  SELECT COALESCE(COUNT(*), 0), COALESCE(SUM(total), 0) INTO p_total_ventas, p_monto_total FROM ventas WHERE id_cliente = p_id;
END$$
DELIMITER ;

CALL resumen_cliente(1, @n, @m);
SELECT @n AS total_ventas;
SELECT @m AS monto_total;
```

- 5) Enunciado: Primero crear una función tiene_saldo_suficiente(p_id INT, p_monto DECIMAL) que retorne TRUE/FALSE según si el saldo del cliente alcanza. Luego crear un procedure transferir_entre_clientes(p_origen INT, p_destino INT, p_monto DECIMAL) que use esa función para validar antes de ejecutar la transferencia, dentro de una transacción completa con manejo de errores.

```
DELIMITER $$
CREATE FUNCTION tiene_saldo_suficiente(
p_id INT,
p_monto DECIMAL(12,2)
)
RETURNS BOOLEAN
READS SQL DATA
BEGIN
    DECLARE variable_saldo DECIMAL(12,2);
    SELECT saldo INTO variable_saldo FROM clientes WHERE id_cliente = p_id;

    IF variable_saldo >= p_monto THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END$$
DELIMITER ;
```

```

DELIMITER $$
CREATE PROCEDURE transferir_entre_clientes(
  IN p_origen INT,
  IN p_destino INT,
  IN p_monto DECIMAL(12,2)
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
  START TRANSACTION;
  IF NOT tiene_saldo_suficiente(p_origen, p_monto) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Saldo Insuficiente';
  END IF;

  UPDATE clientes SET saldo = saldo - p_monto WHERE id_cliente = p_origen;
  UPDATE clientes SET saldo = saldo + p_monto WHERE id_cliente = p_destino;
  COMMIT;
END$$
DELIMITER ;

-- Para corroborar el rollback
CALL transferir_entre_clientes(1, 2, 50000);

```